

Continuous Deployment

Instructor

Kartik Nighania

MLOps Engineer at Typewise

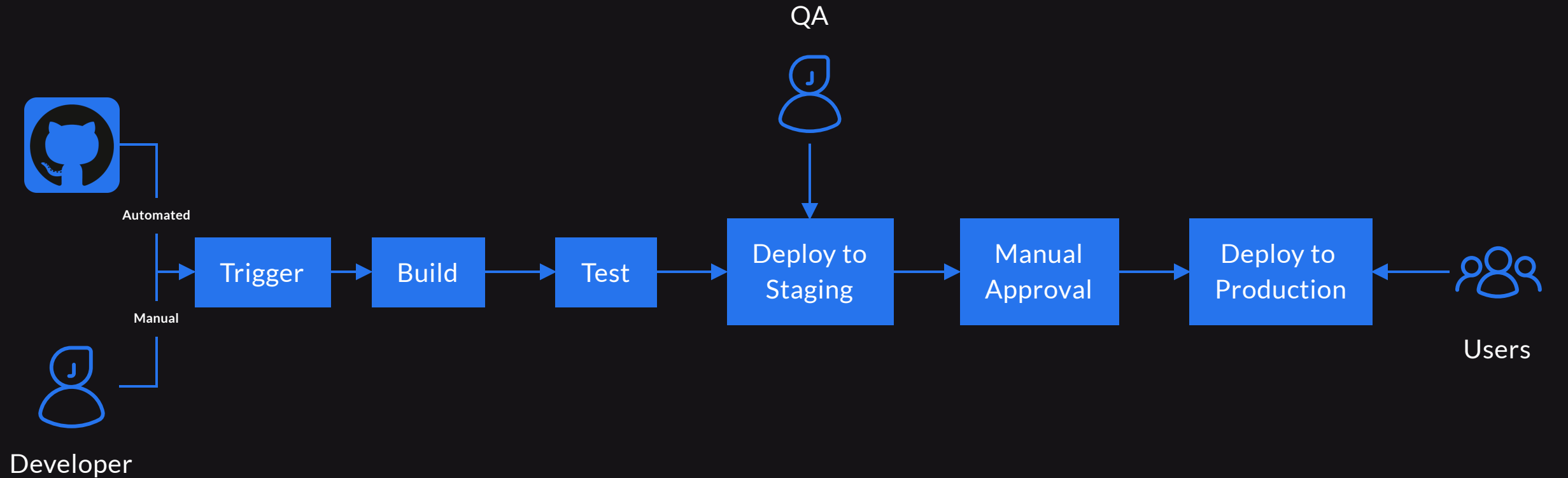
Ex-Head of Engineering - YC-backed startup

Certified AWS Cloud and Kubernetes Engineer

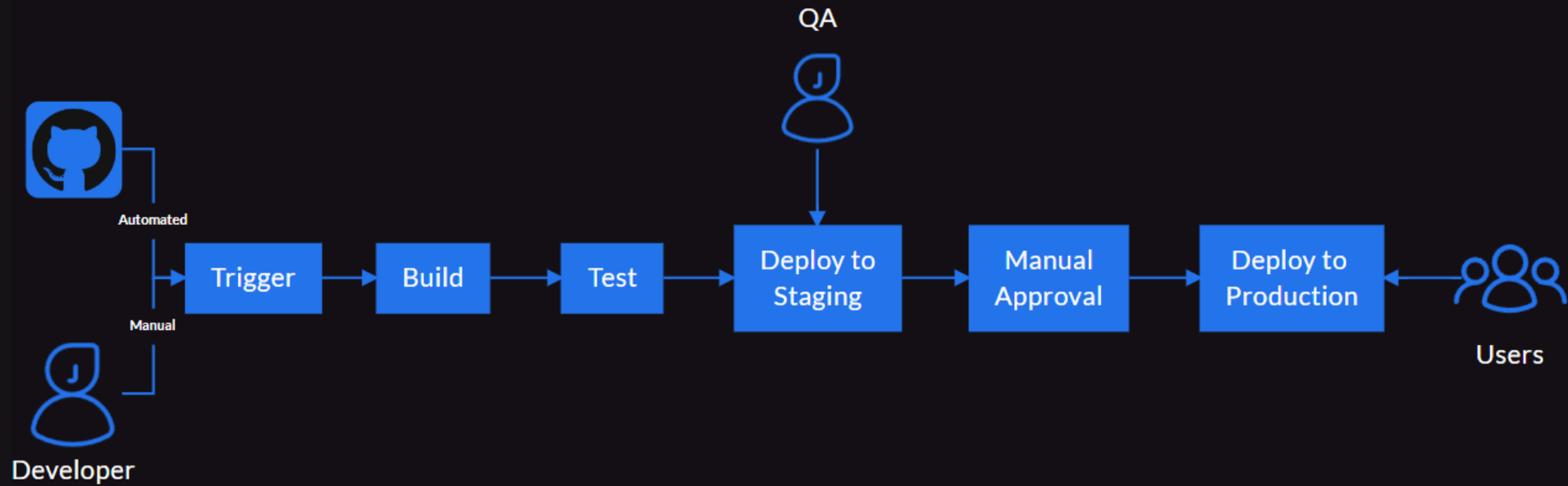
Open source contributor



Continuous Deployment

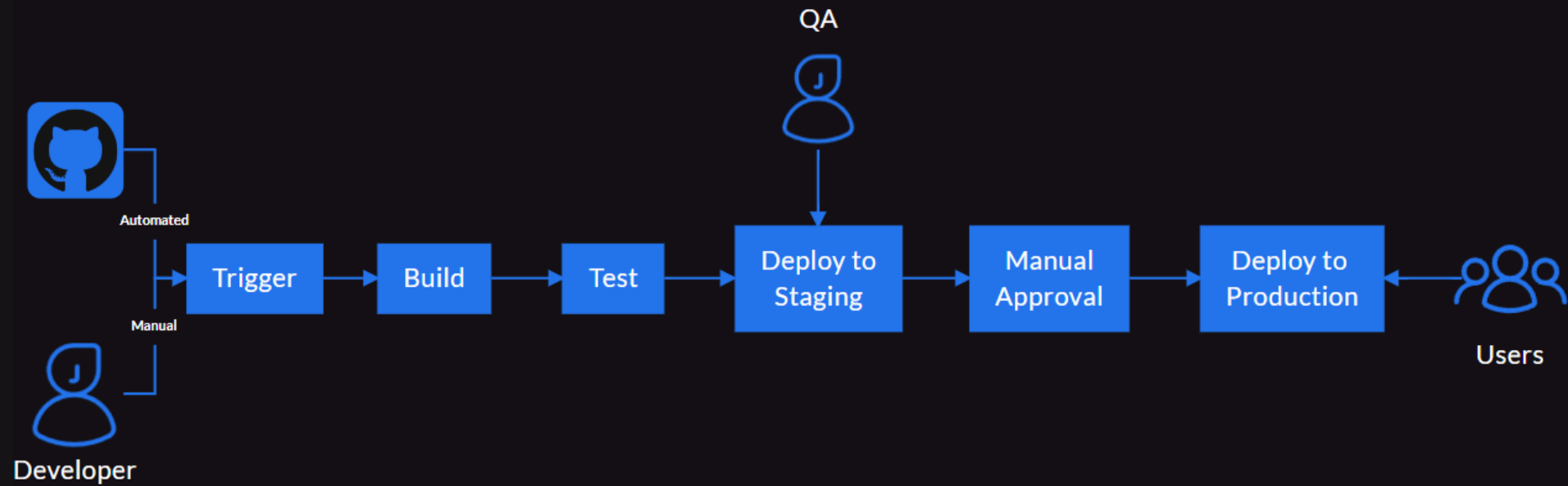


Continuous Deployment



- Code changes to an application are released automatically into the production environment
- Generally, there are 3 stages
 - Development: between multiple teams
 - Staging/UAT: simulate an internal prod environment
 - Production: open to actual users

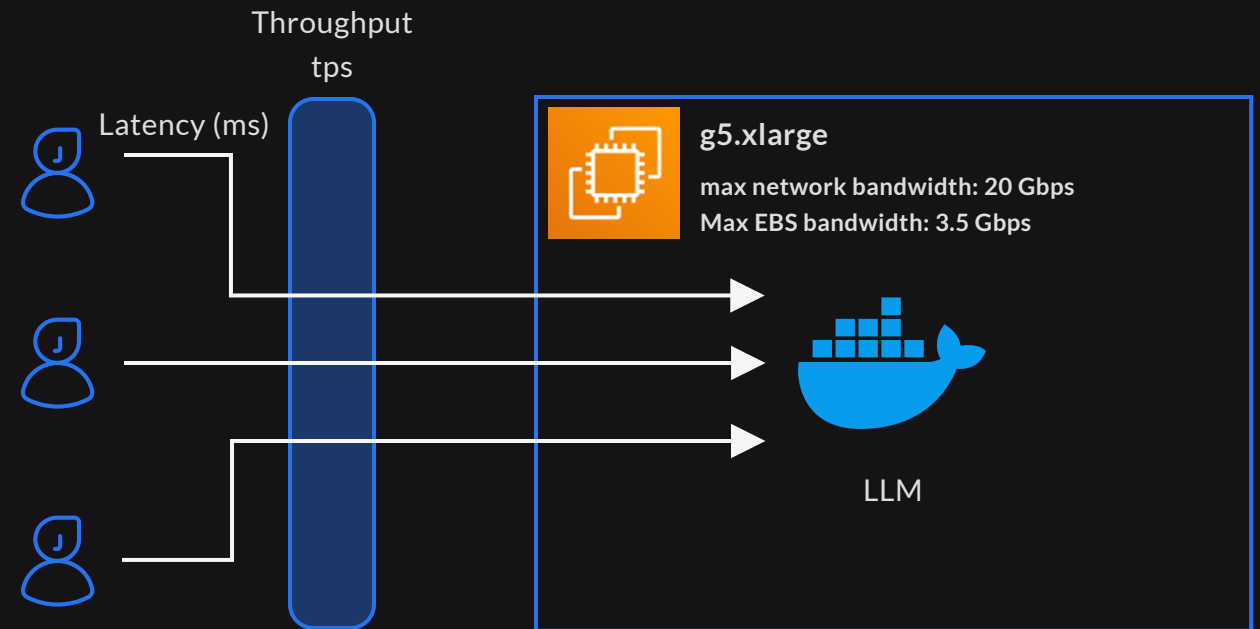
Continuous Deployment



- Application follows versioning system like semantic versioning.
- Semantic versioning consists of major, minor and patch version. For eg: 3.2.1
- Major versions can lead to breaking changes whereas, updates are put in minor changes. Security and bug fixes are usually added in patch version.

Understanding LLM Inference

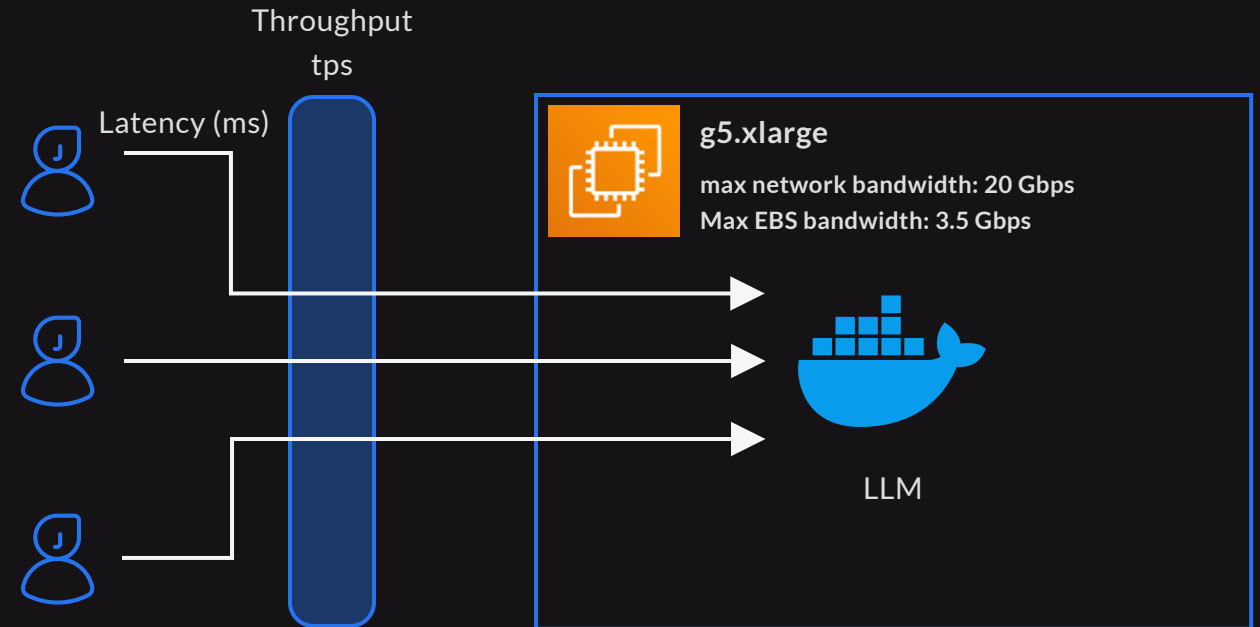
- Models take tokens as input
- Latency vs throughput vs bandwidth
- Throughput depends on the number of concurrent users and is measured in tokens per second (tps)
- Higher the concurrent users higher the latency.



Understanding LLM Inference

LLM inference (TTFT vs TPS):

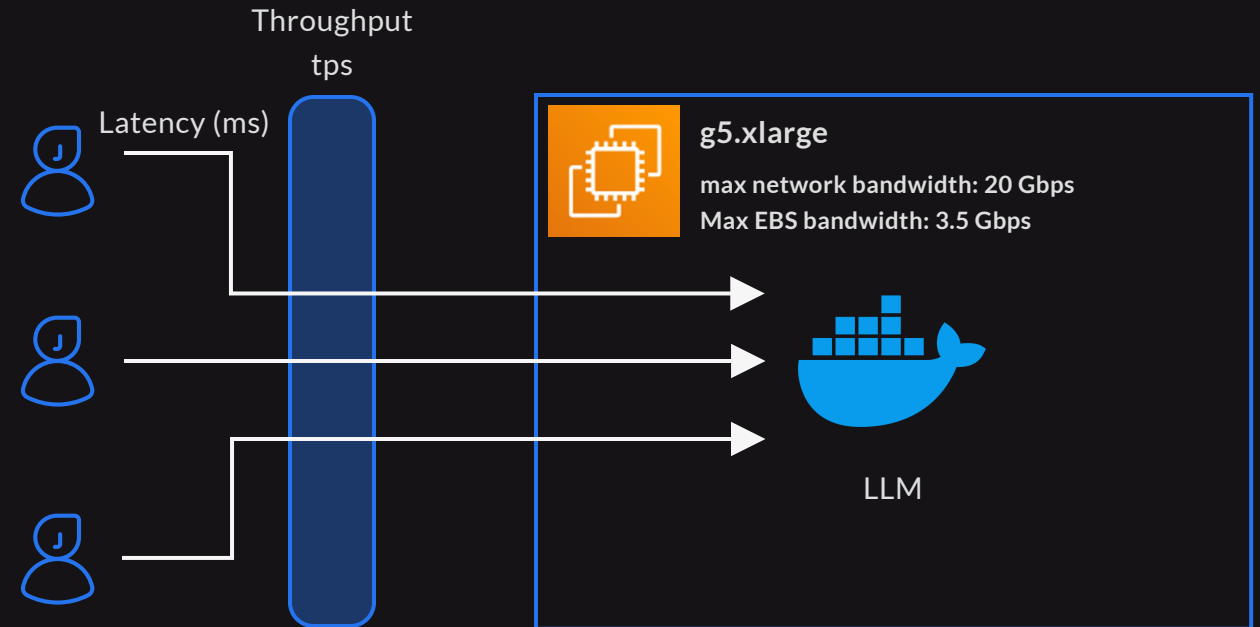
- Prefill phase: compute heavy
- Decode phase: memory transfer overhead. Higher batch size is better



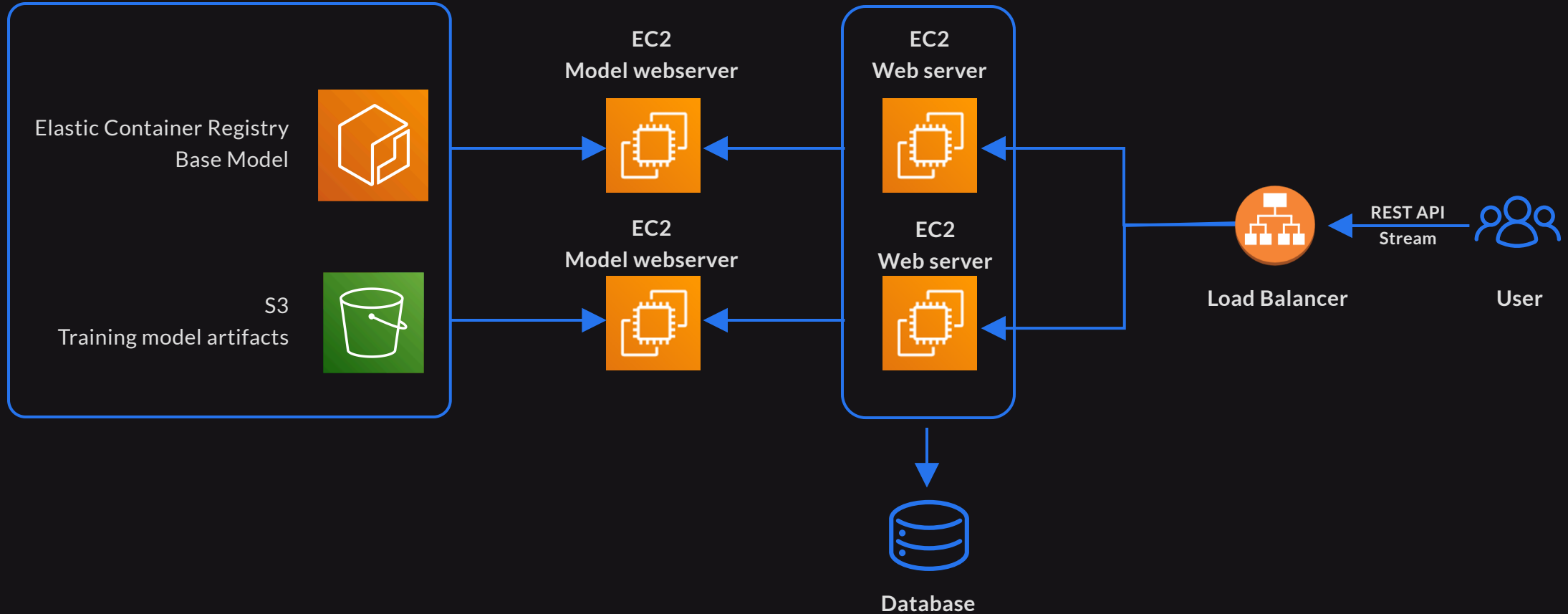
Understanding LLM Inference

LLM inference memory

- Model weights: parameter weights
- KV cache size: Depends on batch size.
Few GBs in size

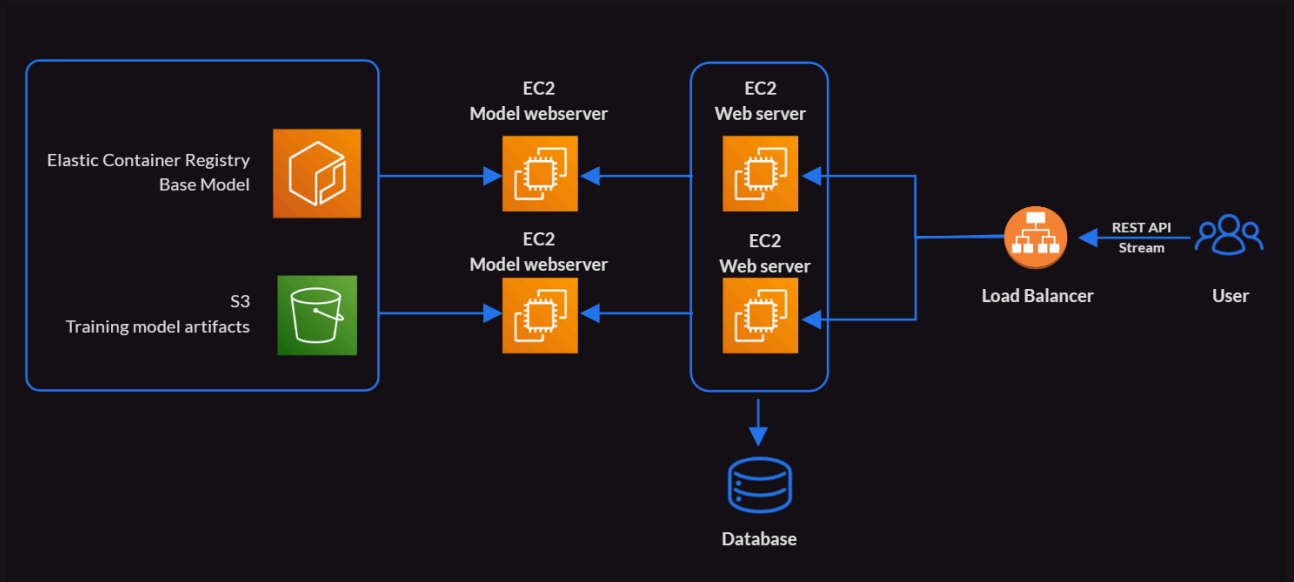


LLM as a Service



LLM as a Service

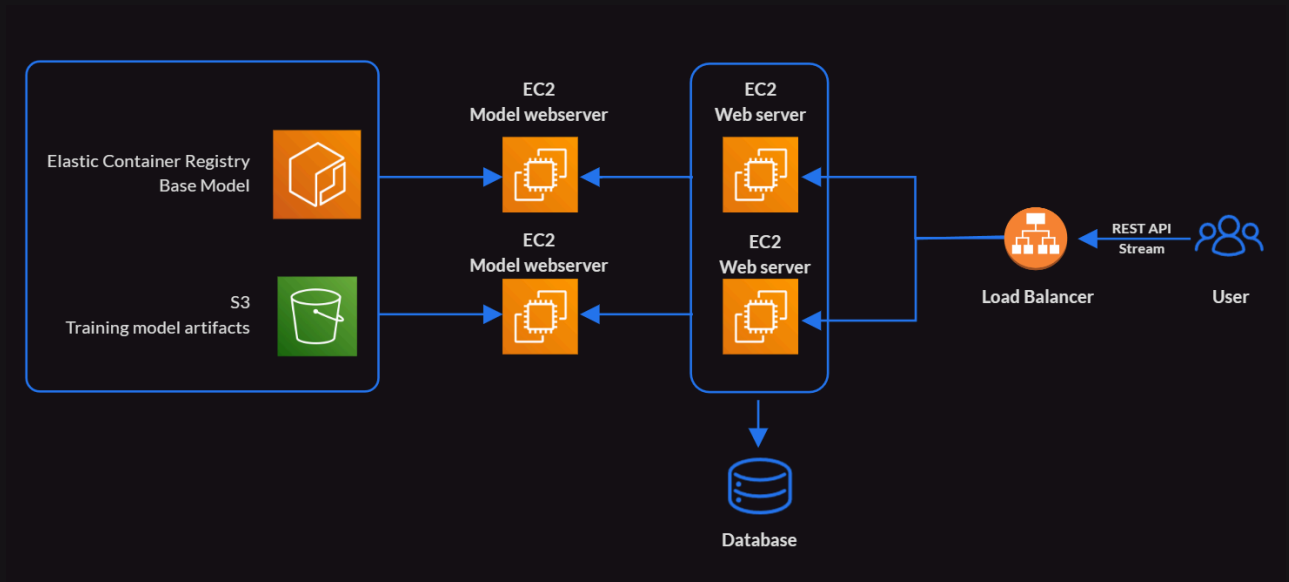
- General use case: REST APIs with support for streaming
- Models are mostly on a GPU backed VMs
- Load balancer evenly distributes load to all web servers and automatically adds them as they scale



LLM as a Service

Advantages of having a middleware service:

- LLM frameworks don't have web-app security built-in
- Separately scale model vs main application
- Control client-facing API structure
- Integrate with tools like databases and vector stores
- Log metrics and monitor with tools like Langfuse



LLM Optimization

Base model Optimization

GQA Grouped Query Attention

- Less attention weights than multi head attention (MHA) For e.g. Llama 3

Quantization

- Decrease memory and computation fp32 -> fp16 -> bf16
- Bf16 has same dynamic range as fp32
- bitsandbytes:
 - model to 8 and 4-bit
 - Outliers processed with fp16
 - On the fly quantization
- AWQ: Activation-Aware Quantization
 - Pre-compute and deploy
 - Important params preserved based on weight activation
 - Faster and less performance degradation

LLM Optimization

KV cache Optimization

Flash attention

- Fusing multiple kernels
- Effective use of GPU memory layers

Pagination of KV cache

- Saving KV cache in noncontiguous space in memory
- Saves wasted slots due to fixed storage reservation on context length per request

LLM Optimization

Request level Optimization

Static Batching

- Multiple request processed by the model at every inference step
- All request prefill and decode together
- Latency vs throughput tradeoff

Continuous Batching

- Token level batching
- Easy swap-in new request tokens
- Swap-out token and return request

Deployment Types: APIs



Ollama

- Local testing of open models
- Quick setup
- Interact with CLI and REST APIs



AWS Bedrock

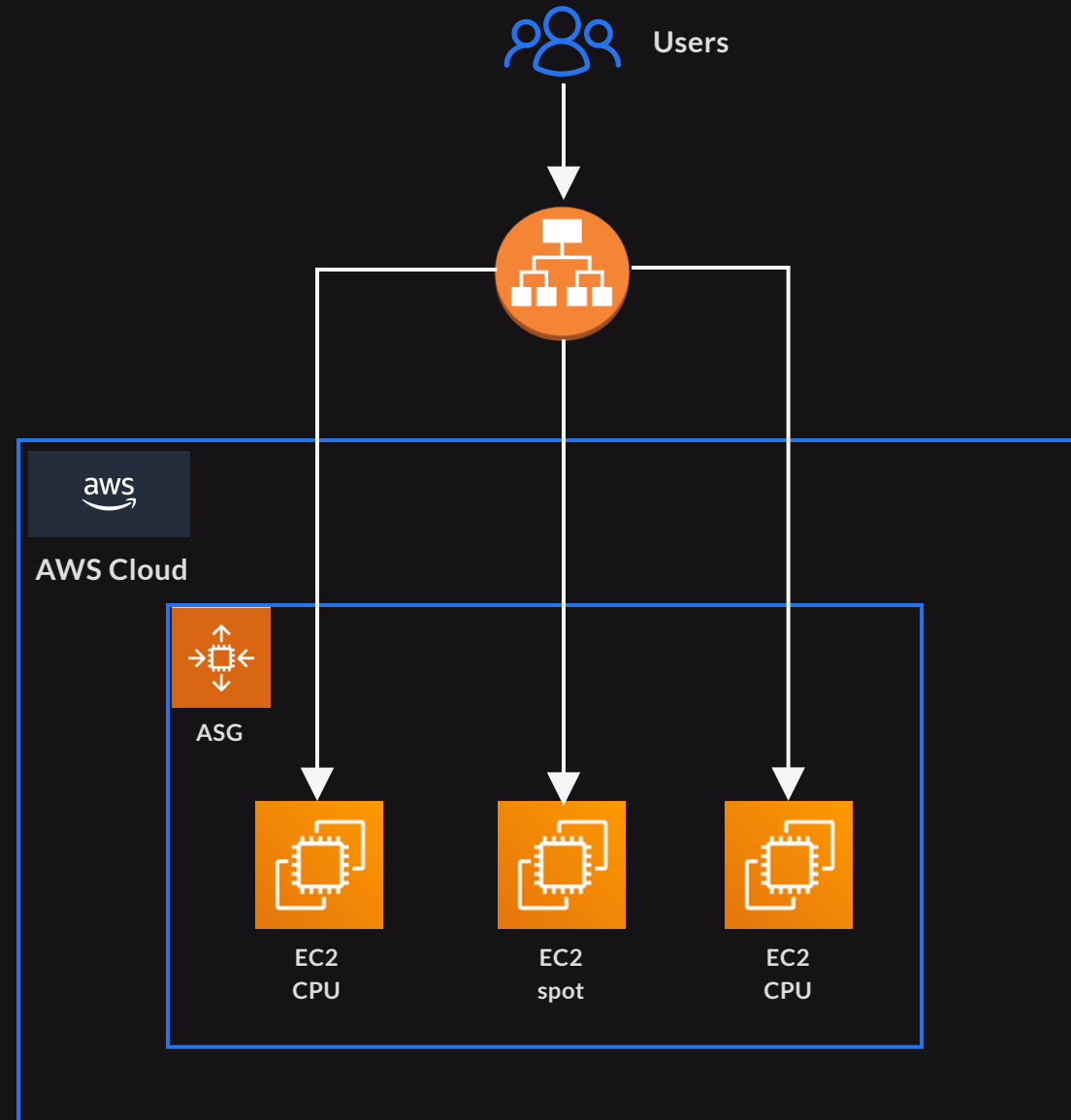
- Pay per use foundation models
- Provisioned throughput options for high-usage
- Possibility to train foundation models
- Region specific service for data boundary



Third Party Providers

- Pay per use model
- Data boundary not defined
- Possibility to fine-tune models

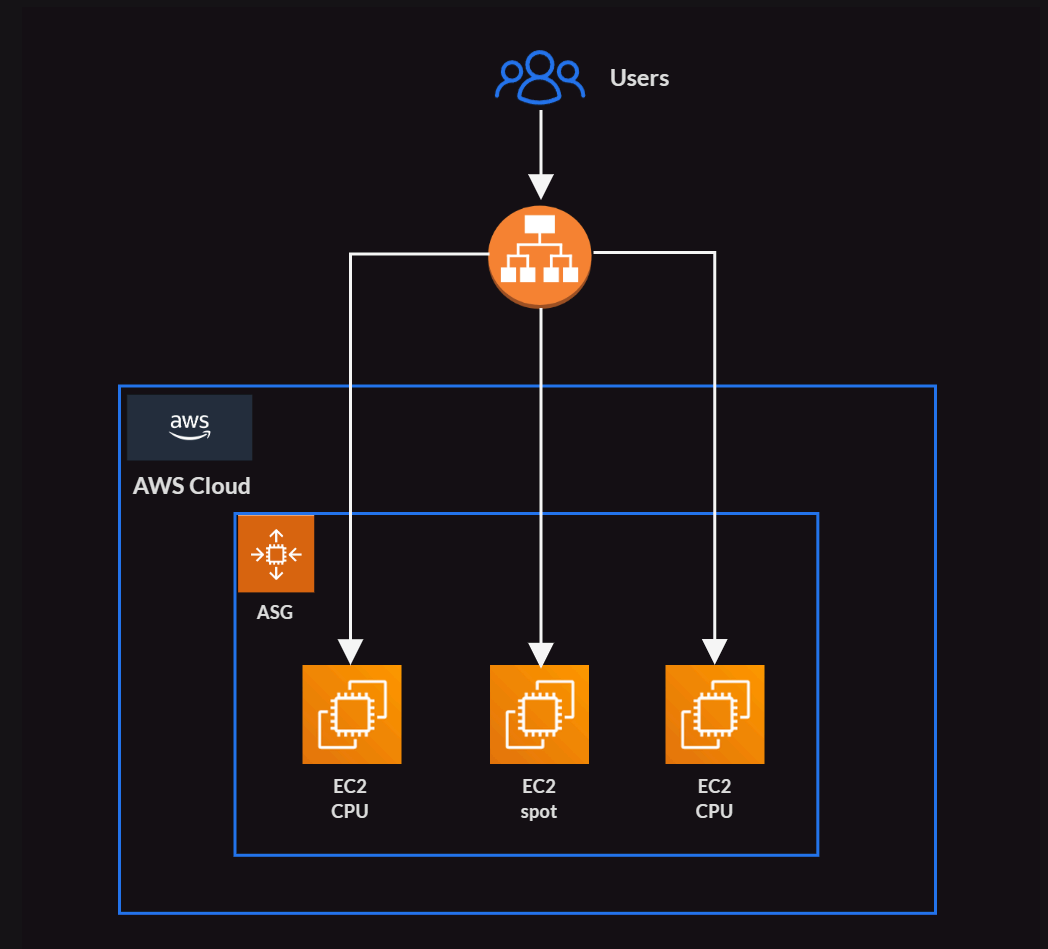
Bare Metal Servers



Bare Metal Servers

Server based deployments

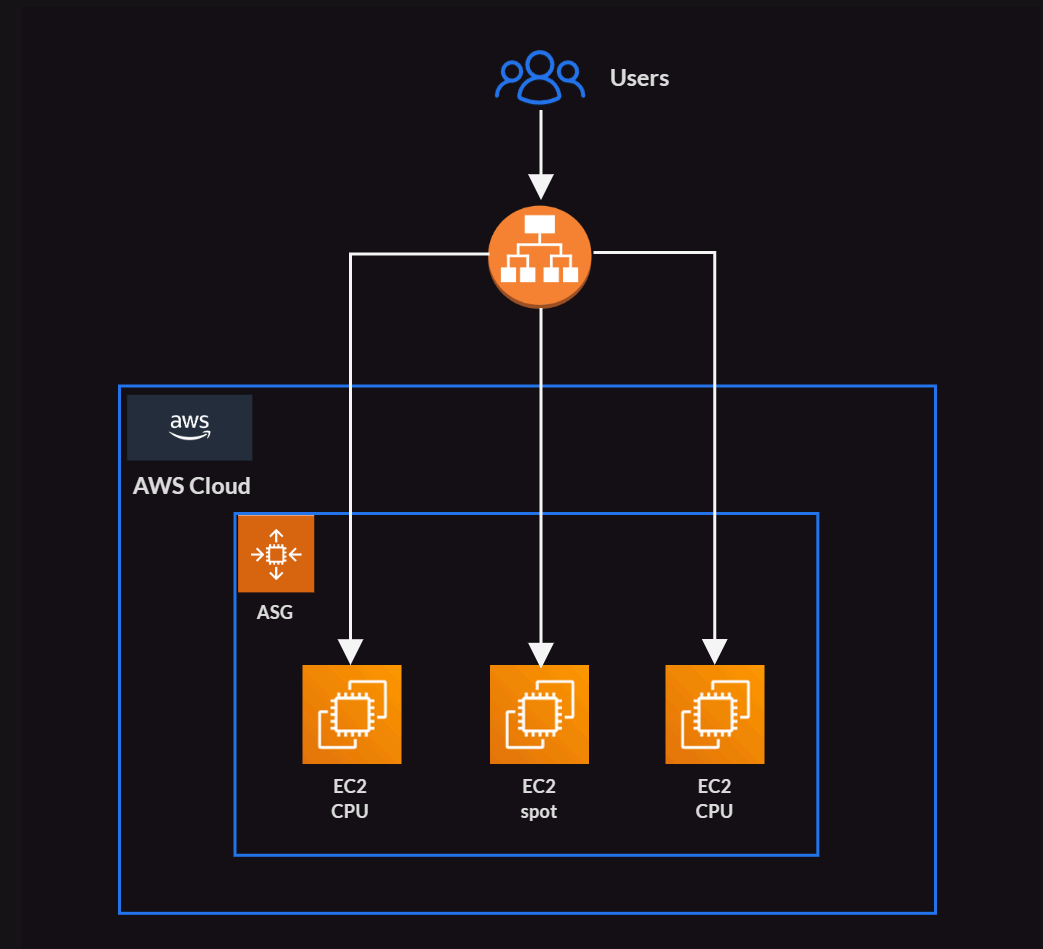
- Use custom VM images having webserver installed
- Startup scripts to install packages and run the application
- Automatic scaling based on metrics like CPU and memory
- Spot instances
 - Typically 70-90% cheaper
 - Stateless application and Client side retry needed



Bare Metal Servers

Server based deployments

- Mixed Instance type making it possible to configure combination of on-demand and spot instances
- Paradigm exists in GCP and Azure as well
- Easy to set and migrate to other clouds
- CPU vs GPU vs TPU vs LPU



Container Deployment Solutions

Elastic Beanstalk

- Platform as a Service
- Easy learning curve
- Automatically handles:
 - capacity provisioning
 - load balancing
 - auto-scaling
 - application monitoring



Container Deployment Solutions

Elastic Kubernetes Service

- Infrastructure as a Service
- Open-sourced version
- Steep learning curve
- More control over infrastructure
- Supports many third-party plugins



Container Deployment Solutions

Elastic Container Service

- Infrastructure as a Service
- Proprietary technology
- Steep learning curve
- More control over infrastructure



Container Deployment Solutions

SageMaker Endpoints

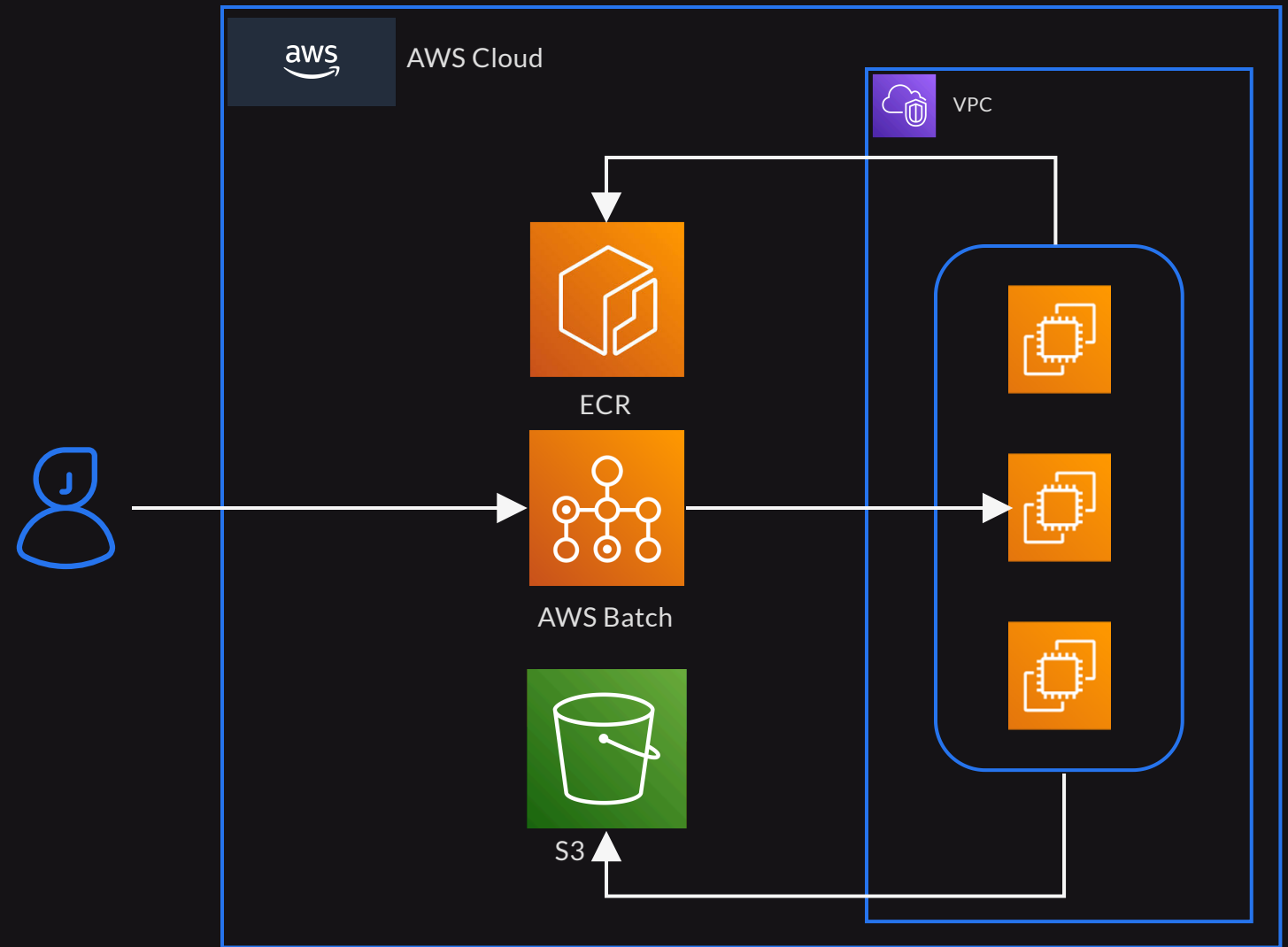
- Platform as a Service
- Easy learning curve
- Default support for multiple LLM models



Async Processing

AWS Batch

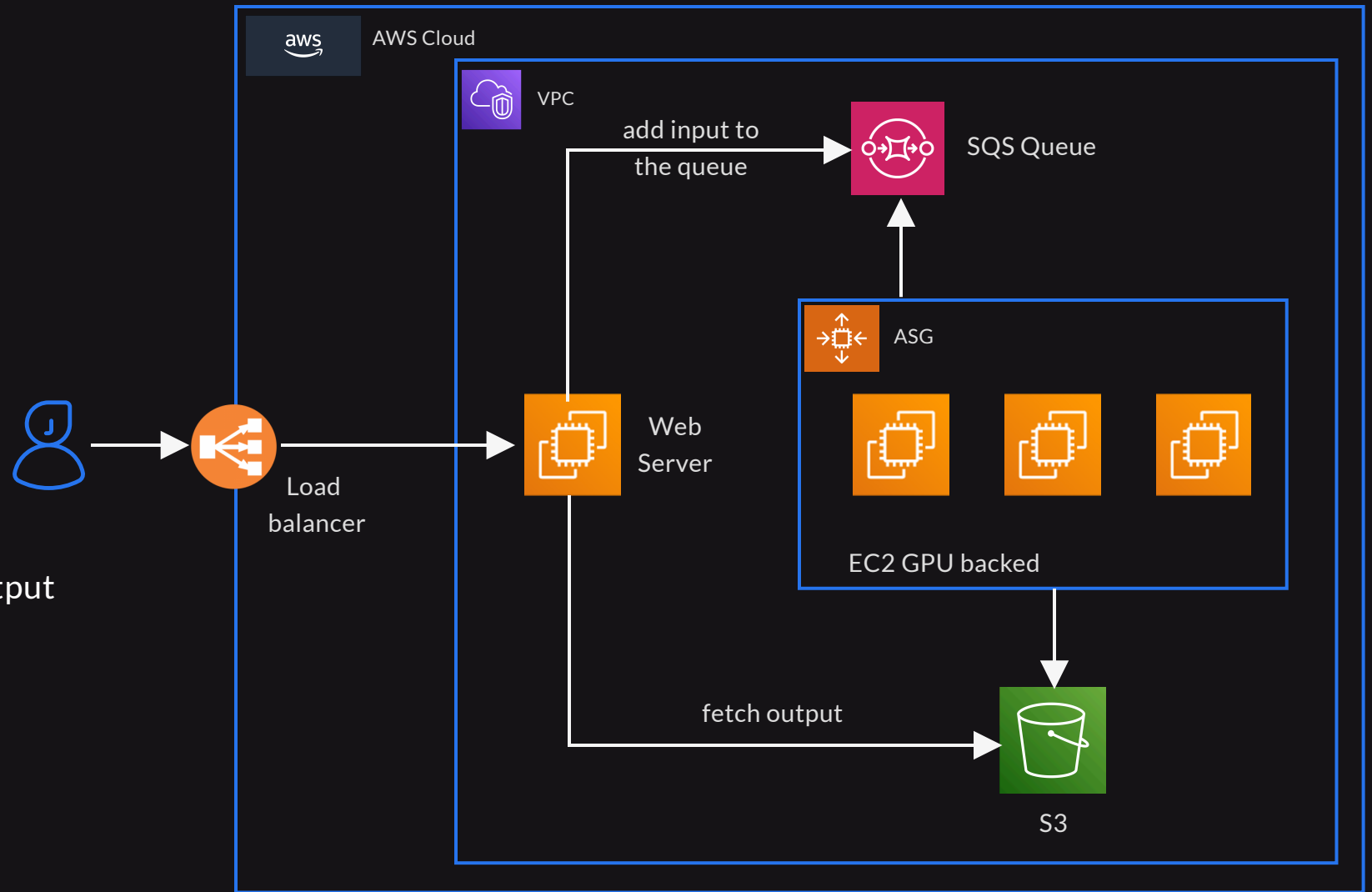
- Launch containers using EC2, Fargate or EKS
- Scaling handled by AWS batch
- Resources removed after task completion
- Support for trigger events



Async Processing

Asynchronous APIs

- Send request to queue for processing
- Use a queue to decouple requests
- Separately scale model EC2 instances
- Client side code retires to check for output



Thank You
